E-Grocery AI Agent: Deployment & Code Guide

Production-Ready Implementation using Streamlit & Snowflake Cortex AI

TARGET DB	EMBEDDING MODEL	CORTEX MODELS	UI DEPLOYMENT
E_GROCERY_DB	e5-base-v2 (768-dim)	Claude 3.5 / Llama 3.1	Streamlit in Snowflake

1. ARCHITECTURE PREREQUISITES & SETUP

Before executing the deployment scripts, ensure your Snowflake account has ACCOUNTADMIN or equivalent role access to create databases, warehouses, stages, and enable Cortex LLM cross-region evaluation.

2. STEP 1: DATABASE, SCHEMA & TABLE DDL

Execute this script in a Snowflake SQL Worksheet to set up the infrastructure, product catalog, customer orders, policy chunk vector store, and sentiment logs:

sql/01_setup_database_and_tables.sql

```
-- ================================================================== -- Script: 01_setup_database_
and_tables.sql -- Description: Sets up Database, Schema, Virtual Warehouse, Tables, -- and Internal Staging
for the E-Grocery AI Agent. -- ================================================================== USE ROL
E ACCOUNTADMIN; -- 1. Create Dedicated Warehouse & Database Infrastructure CREATE DATABASE IF NOT EXISTS E_
GROCERY_DB; CREATE SCHEMA IF NOT EXISTS E_GROCERY_DB.SUPPORT_SCHEMA; CREATE WAREHOUSE IF NOT EXISTS E_GROCE
RY_WH WAREHOUSE_SIZE = 'XSMALL' AUTO_SUSPEND = 60 AUTO_RESUME = TRUE; USE DATABASE E_GROCERY_DB; USE SCHEMA
SUPPORT_SCHEMA; USE WAREHOUSE E_GROCERY_WH; -- Enable Cross-Region Cortex evaluation if using non-US defaul
t regions ALTER ACCOUNT SET CORTEX_ENABLED_CROSS_REGION = 'ANY_REGION'; -- 2. Create Product Catalog Table
(Inventory & Dietary Allergens) CREATE OR REPLACE TABLE PRODUCT_CATALOG ( PRODUCT_ID STRING, PRODUCT_NAME S
TRING, CATEGORY STRING, PRICE FLOAT, STOCK_QTY INT, DIETARY_INFO STRING, SPECIFICATIONS STRING ); INSERT IN
TO PRODUCT_CATALOG VALUES ('P001', 'Organic Whole Milk 1L', 'Dairy', 3.49, 120, 'Organic, Pasteurized, Cont
ains Dairy', 'Keep refrigerated at 4C'), ('P002', 'Gluten-Free Almond Bread', 'Bakery', 5.99, 45, 'Gluten-F
ree, Vegan, Contains Nuts', 'Made with organic almond flour'), ('P003', 'Fresh Hass Avocados (Pack of 4)',
'Produce', 4.99, 85, 'Vegan, Non-GMO', 'Ripe Hass avocados from Mexico'), ('P004', 'Organic Honeycrisp Appl
es 2lb', 'Produce', 6.49, 10, 'Organic, Fresh Fruit', 'Sweet crisp apples'); -- 3. Create Customer Transact
ional Orders Table CREATE OR REPLACE TABLE CUSTOMER_ORDERS ( ORDER_ID STRING, CUSTOMER_ID STRING, PRODUCT_N
AME STRING, CATEGORY STRING, ORDER_DATE TIMESTAMP_NTZ, STATUS STRING, TOTAL_AMOUNT FLOAT ); INSERT INTO CUS
TOMER_ORDERS VALUES ('ORD-9001', 'CUST-101', 'Fresh Hass Avocados (Pack of 4)', 'Produce', DATEADD('hour',
-12, CURRENT_TIMESTAMP()), 'Delivered', 4.99), ('ORD-9002', 'CUST-101', 'Organic Whole Milk 1L', 'Dairy', D
ATEADD('day', -3, CURRENT_TIMESTAMP()), 'Delivered', 3.49), ('ORD-9003', 'CUST-102', 'Gluten-Free Almond Br
ead', 'Bakery', CURRENT_TIMESTAMP(), 'Out for Delivery', 5.99); -- 4. Create Vector Store Table for Policy
Documents CREATE OR REPLACE TABLE POLICY_DOCUMENT_CHUNKS ( DOC_NAME STRING, CHUNK_ID NUMBER AUTOINCREMENT,
CHUNK_TEXT STRING, CHUNK_EMBEDDING VECTOR(FLOAT, 768), UPDATED_AT TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
); -- 5. Create Customer Feedback & Cortex Sentiment Audit Log CREATE OR REPLACE TABLE CUSTOMER_FEEDBACK (
FEEDBACK_ID NUMBER AUTOINCREMENT, CUSTOMER_ID STRING, ORDER_ID STRING, MESSAGE STRING, CATEGORY STRING, SEN
TIMENT_SCORE FLOAT, LOGGED_AT TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP() ); -- 6. Create Internal Stage for
Policy Uploads CREATE OR REPLACE STAGE ONLINE_POLICY_STAGE DIRECTORY = (ENABLE = TRUE) ENCRYPTION = (TYPE =
'SNOWFLAKE_SSE');
```

3. STEP 2: POLICY DOCUMENT INGESTION & VECTOR EMBEDDING

This script processes markdown files stored in `@ONLINE_POLICY_STAGE`, splits them into semantic chunks, and calculates 768-dimensional embeddings using Cortex `e5-base-v2`:

sql/02_ingest_and_embed_policies.sql

```
-- ================================================================== -- Script: 02_ingest_and_embed_policies.sql -- Description: Parses uploaded Markdown policies, splits text into -- chunks, and computes Cortex Vector Embeddings. -- ================================================================== USE DATABASE E_GROCERY_DB; USE SCHEMA SUPPORT_SCHEMA; USE WAREHOUSE E_GROCERY_WH; -- Option A: Automated Chunking & Vectorization from Internal Stage INSERT INTO POLICY_DOCUMENT_CHUNKS (DOC_NAME, CHUNK_TEXT, CHUNK_EMBEDDING) WITH PARSED_DOCS AS ( SELECT RELATIVE_PATH AS DOC_NAME, SNOWFLAKE.CORTEX.PARSE_DOCUMENT(@ONLINE_POLICY_STAGE, RELATIVE_PATH, {'mode': 'LAYOUT'}) AS RAW_CONTENT FROM DIRECTORY(@ONLINE_POLICY_STAGE) ), TEXT_CHUNKS AS ( SELECT DOC_NAME, CHUNK.VALUE::STRING AS CHUNK_TEXT FROM PARSED_DOCS, LATERAL FLATTEN( INPUT => SNOWFLAKE.CORTEX.SPLIT_TEXT_RECURSIVE_CHARACTER( RAW_CONTENT:content::STRING, 'markdown', 800, 100 ) ) CHUNK ) SELECT DOC_NAME, CHUNK_TEXT, SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', CHUNK_TEXT) AS CHUNK_EMBEDDING FROM TEXT_CHUNKS; -- Option B: Direct Fallback Vector Seed Insertion (Guarantee Data Availability) INSERT INTO POLICY_DOCUMENT_CHUNKS (DOC_NAME, CHUNK_TEXT, CHUNK_EMBEDDING) SELECT 'E_GROCERY_ONLINE_SHOPPING_POLICIES.md', 'Freshness & Quality Guarantee: We guarantee 100% freshness on all perishable products including organic dairy, fresh produce, meat, and baked goods. If any item delivered is expired, bruised, rotten, or spoiled, customers are eligible for an instant store credit or full refund if reported within 48 hours of delivery.', SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', 'Freshness & Quality Guarantee: We guarantee 100% freshness on all perishable products including organic dairy, fresh produce, meat, and baked goods. If any item delivered is expired, bruised, rotten, or spoiled, customers are eligible for an instant store credit or full refund if reported within 48 hours of delivery.'); INSERT INTO POLICY_DOCUMENT_CHUNKS (DOC_NAME, CHUNK_TEXT, CHUNK_EMBEDDING) SELECT 'E_GROCERY_ONLINE_SHOPPING_POLICIES.md', 'Order Cancellations & Modifications: Orders can be modified or canceled free of charge up to 2 hours before the scheduled delivery window. Once an order enters the "Out for Delivery" state, cancellations incur a $5.00 restocking fee.', SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', 'Order Cancellations & Modifications: Orders can be modified or canceled free of charge up to 2 hours before the scheduled delivery window. Once an order enters the "Out for Delivery" state, cancellations incur a $5.00 restocking fee.');
```

💡 **Production Tip:** For continuous ingestion, create a **Snowflake Task** triggered on `ONLINE_POLICY_STAGE` directory updates to auto-embed newly added policy files.

4. STEP 3: STREAMLIT IN SNOWFLAKE (SIS) APPLICATION CODE

Deploy this code directly inside the Streamlit editor in Snowflake to run the interactive support interface:

streamlit_app/app.py

```
# ====================================================================== # Application: app.py # Description: Streamlit in Snowflake (SiS) Production Chat Application # ========================================================================== import streamlit as st import pandas as pd from snowflake.snowpark.context import get_active_session st.set_page_config(page_title="E-Grocery Support Agent", page_icon="🛒", layout="wide") session = get_active_session() # Sidebar Customer Context st.sidebar.title("👤 Active Session") customer_id = st.sidebar.text_input("Customer ID:", value="CUST-101") llm_model = st.sidebar.selectbox("Cortex Engine:", ("claude-3-5-sonnet", "llama3.1-70b", "mistral-large2"), index=0) st.title("🛒 E-Grocery AI Support Agent") st.caption("Powered by Snowflake Cortex AI Vector Search & Grounded RAG") if "messages" not in st.session_state: st.session_state.messages = [] for msg in st.session_state.messages: with st.chat_message(msg["role"]): st.markdown(msg["content"]) def search_policy_documents(query_text, top_k=3): clean_q = query_text.replace("'", "''") vector_sql = f""" WITH query_vec AS ( SELECT SNOWFLAKE.CORTEX.EMBED_TEXT_768('e5-base-v2', '{clean_q}') AS q_emb ) SELECT DOC_NAME, CHUNK_TEXT, VECTOR_COSINE_SIMILARITY(CHUNK_EMBEDDING, q_emb) AS SIMILARITY_SCORE FROM POLICY_DOCUMENT_CHUNKS, query_vec ORDER BY SIMILARITY_SCORE DESC LIMIT {top_k} """ return session.sql(vector_sql).to_pandas() def analyze_and_log_feedback(cust_id, msg_text): clean_msg = msg_text.replace("'", "''") score = float(session.sql(f"SELECT SNOWFLAKE.CORTEX.SENTIMENT('{clean_msg}') AS S").to_pandas().iloc[0]['S']) cat = session.sql(f"SELECT SNOWFLAKE.CORTEX.COMPLETE('llama3.1-8b', 'Categorize into [Delivery, Return, Quality]: {clean_msg}') AS C").to_pandas().iloc[0]['C'].strip() session.sql(f"INSERT INTO CUSTOMER_FEEDBACK (CUSTOMER_ID, ORDER_ID, MESSAGE, CATEGORY, SENTIMENT_SCORE) VALUES ('{cust_id}', 'N/A', '{clean_msg}', '{cat}', {score})").collect() return score if user_input := st.chat_input("Ask about delivery rules, spoiled items, or refunds..."): st.session_state.messages.append({"role": "user", "content": user_input}) with st.chat_message("user"): st.markdown(user_input) with st.chat_message("assistant"): with st.spinner("Analyzing query & retrieving policy context..."): s_score = analyze_and_log_feedback(customer_id, user_input) policies_df = search_policy_documents(user_input, top_k=3) orders_df = session.sql(f"SELECT * FROM CUSTOMER_ORDERS WHERE CUSTOMER_ID='{customer_id}'").to_pandas() catalog_df = session.sql("SELECT * FROM PRODUCT_CATALOG").to_pandas() policy_ctx = "\n\n".join([f"[{r['DOC_NAME']}]: {r['CHUNK_TEXT']}" for _, r in policies_df.iterrows()]) prompt = f"""You are an E-Grocery AI Support Agent. Customer ID: {customer_id} | Sentiment Score: {s_score} Policies Context:\n{policy_ctx}\n Product Catalog:\n{catalog_df.to_string()}\n Customer Orders:\n{orders_df.to_string()}\n Customer Query: {user_input} Instructions: Provide a helpful, direct response citing policy names. If sentiment < -0.5, offer immediate human escalation.""" resp = session.sql(f"SELECT SNOWFLAKE.CORTEX.COMPLETE('{llm_model}', '{prompt.replace(chr(39), chr(39)+chr(39))}') AS R").to_pandas().iloc[0]['R'] st.markdown(resp) with st.expander("📄 View Grounding Context & Vector Match Scores"): st.dataframe(policies_df[['DOC_NAME', 'SIMILARITY_SCORE', 'CHUNK_TEXT']]) st.session_state.messages.append({"role": "assistant", "content": resp})
```

5. STEP 4: CONFIGURATION FILES

Include these configuration files when bundling or deploying the application package:

`streamlit_app/environment.yml`

```
name: egrocery_cortex_app channels: - snowflake dependencies: - python=3.10 - streamlit - snowflake-snowpark-python - pandas
```

`requirements.txt`

```
snowflake-snowpark-python streamlit pandas
```

6. EXECUTION & VERIFICATION WORKFLOW

Follow this exact sequence to deploy and test the entire solution in Snowflake:

- **Step 6.1 (Run DDL Script):** Execute `sql/01_setup_database_and_tables.sql` in a Snowflake SQL Worksheet under the `ACCOUNTADMIN` role.
- **Step 6.2 (Upload Documents):** In Snowflake Snowsight UI, navigate to **Data » Databases » E_GROCERY_DB » SUPPORT_SCHEMA » Stages » ONLINE_POLICY_STAGE** and upload your `.md` policy files.
- **Step 6.3 (Vector Generation):** Execute `sql/02_ingest_and_embed_policies.sql` to chunk and compute `e5-base-v2` embeddings.
- **Step 6.4 (Launch Streamlit App):** Go to **Projects » Streamlit**, create a new app named `E_GROCERY_SUPPORT_APP`, set the database to `E_GROCERY_DB`, schema to `SUPPORT_SCHEMA`, paste `streamlit_app/app.py` into the editor, and click **Run**.
- **Step 6.5 (Verification):** Submit test queries such as *"Can I get a refund for bruised avocados?"* or *"Do you have gluten-free bread in stock?"* and check the expandable drawer to verify citation source mapping and sentiment score logging in `CUSTOMER_FEEDBACK`.

> 🛡️ **Security Verification:** Validate that all queries run under role-scoped warehouse execution and that no session data leaves the Snowflake tenant perimeter.